Artificial Intelligence Digital Assignment(Case Study) (10 Marks)

Interactive 8-Puzzle Problem Solver using AI Search Algorithms

Course: Artificial Intelligence

Individual Assignment

Submission: GitHub Repository + GitHub Pages Live URL

Objective

The objective of this assignment is to understand the working of **state-space search algorithms** by developing an interactive web application that solves the **8-Puzzle Problem** using both uninformed and informed search techniques.

Students are expected to implement the search algorithms from scratch using JavaScript and visually demonstrate how each algorithm searches for the goal state.

Problem Statement

Develop an interactive web application that solves the **8-Puzzle Problem** using **any one** of the following search algorithms. Each student is assigned **one specific algorithm**, and the algorithm allotted to each student is listed **at the end of this document**. Students must implement **only the algorithm assigned to them**.

Uninformed Search Algorithms

1. Breadth First Search (BFS)
2. Depth First Search (DFS)
3. Depth Limited Search (DLS)
4. Iterative Deepening Depth Limited Search (IDDLS)
5. Uniform cost search

Informed Search Algorithms

6. Greedy Best First Search (GBFS)
7. A* Search

Technologies

Students must use

- HTML5
- CSS3
- JavaScript (ES6)

Any one UI Framework

- Bootstrap
  OR
- Tailwind CSS

Optional

- React
- Vue
- Angular

Animation Libraries (Optional)

- Canvas API
- SVG
- D3.js

Note: The search algorithms can implement either manually or usage of AI/search libraries for algorithm implementation.

Website Requirements

The application should contain the following pages/features.

---

1. Home Page

The homepage should display

- Course Name
- Assignment Title
- Student Name
- Registration Number
- Introduction to the 8-Puzzle Problem
- Navigation Menu

---

2. Puzzle Input

Provide two methods to create the puzzle.

Method 1

User enters

- Initial State

- Goal State

Example

Initial State

2 8 3

1 6 4

7 _ 5

Goal State

1 2 3

8 _ 4

7 6 5

Method 2

Generate a random solvable puzzle.

Provide a button

Generate Random Puzzle

---

3. Algorithm Selection

The user should be able to choose

- BFS
- DFS
- DLS
- IDDLS
- GBFS
- A*

For DLS

Provide

Depth Limit

For GBFS and A*

Provide heuristic selection.

Students should implement at least the following heuristics.

- Misplaced Tiles
- Manhattan Distance

---

4. Puzzle Visualization

The application should visually display

- Initial State
- Goal State
- Current State
- Generated Successor States
- Final Solution Path

The movement of the blank tile should be animated.

Example

Step 0

2 8 3

1 6 4

7 _ 5

↓

Step 1

2 8 3

1 _ 4

7 6 5

↓

Step 2

2 _ 3

1 8 4

7 6 5

Continue until the goal state is reached.

---

5. Search Process Visualization

During execution, display the following information in real time.

- Current node being expanded
- Number of nodes generated
- Number of nodes expanded
- Frontier (Open List)
- Explored (Closed List)
- Current search depth
- Parent node
- Path cost (if applicable)

For GBFS and A*, also display

- Heuristic value h(n)
- Path cost g(n)
- Evaluation function f(n)

---

6. Solution Summary

After reaching the goal state, display

- Solution Found / Not Found
- Total Number of Moves
- Total Nodes Generated
- Total Nodes Expanded
- Maximum Search Depth
- Execution Time
- Path Cost
- Memory Usage (Approximate)

---

7. Comparison Page

Provide a comparison table showing the performance of all six algorithms on the same puzzle instance.

Example

Algorithm	Solution Found	Nodes Expanded	Moves	Time	Memory
BFS	Yes	152	8	24 ms	High
DFS	Yes	391	21	12 ms	Low
DLS	No	85	-	8 ms	Low
IDDLS	Yes	170	8	30 ms	Medium
GBFS	Yes	39	8	5 ms	Low
A*	Yes	25	8	4 ms	Medium

---

8. Algorithm Information Page

For each algorithm, include

- Description
- Working Principle
- Advantages
- Limitations
- Time Complexity
- Space Complexity
- Completeness
- Optimality

---

9. Interactive Controls

Provide the following controls.

- Run
- Pause
- Resume
- Next Step
- Previous Step
- Reset

- Change Speed

GitHub Repository Structure

AI-8Puzzle-Visualizer/

|— index.html

|— css/

|— js/

|— assets/

|— images/

|— README.md

|— docs/

README.md Should Include

- Project Description
- Technologies Used
- Search Algorithms Implemented
- Heuristics Used
- Screenshots
- GitHub Pages URL
- Student Details
- References
- How to Run the Project

Deployment

Students must deploy the project using **GitHub Pages**.

Submit the following.

1. GitHub Repository Link

Example

https://github.com/username/AI-8Puzzle-Visualizer

2. Live Website

Example

https://username.github.io/AI-8Puzzle-Visualizer

Evaluation Rubric (10 Marks)

Component	**Marks**
Correct implementation of BFS, DFS, DLS and IDDLS	2
Correct implementation of GBFS and A* with heuristics	2
Interactive visualization of the search process	2
User interface, responsiveness and usability	1
Performance statistics, comparison page and algorithm documentation	1
GitHub repository quality, code organization and README	1
Successful deployment on GitHub Pages and project demonstration	1
Total	**10**

Submission Guidelines

Each student must submit the following through the pdf which contains the URL's of the below requirements:

1. GitHub Repository URL
2. GitHub Pages Live URL
3. README.md
4. A **3–5 minute demonstration video** explaining:
    - The implemented algorithms
    - Puzzle solving process
    - Comparison of algorithms
    - Heuristic functions used
    - Overall project features

Academic Integrity

- Properly acknowledge all third-party resources in the README.

Expected Learning Outcomes

Upon successful completion of this assignment, students will be able to:

- Model the **8-Puzzle** as a state-space search problem.
- Implement both uninformed and informed search algorithms from scratch.
- Compare algorithms based on completeness, optimality, time, and memory usage.
- Apply heuristic functions such as **Misplaced Tiles** and **Manhattan Distance**.
- Develop an interactive AI visualization tool using modern web technologies.
- Deploy a complete web application using GitHub and GitHub Pages.

Assignment of Algorithm for Implementation

REGISTER NO	Algorithm Number
24BAI0001	1
24BAI0077	2
24BAI0117	3
24BAI0128	4
24BAI0163	5
24BCB0061	6
24BCB0063	7
24BCB0161	1
24BCE0057	2
24BCE0324	3
24BCE0437	4
24BCE0570	5
24BCE0867	6
24BCE2054	7
24BCE2058	1
24BCE2059	2
24BCE2100	3
24BCE2101	4
24BCE2152	5
24BCE2526	6
24BCE2549	7
24BCE2582	1
24BCE2592	2
24BCE2612	3
24BCE2615	4
24BCE2650	5
24BCE2669	6
24BCE2670	7

24BCE2690	1
24BCE2707	2
24BCE2733	3
24BCE2736	4
24BCE2803	5
24BCE2812	6
24BCE2871	7
24BCE2946	1
24BCI0083	2
24BCI0124	3
24BCI0159	4
24BCI0175	5
24BCI0212	6
24BCI0220	7
24BCT0022	1
24BCT0046	2
24BCT0058	3
24BCT0061	4
24BCT0107	5
24BCT0117	6
24BCT0292	7
24BDS0004	1
24BDS0071	2
24BDS0105	3
24BDS0111	4
24BDS0130	5
24BDS0131	6
24BDS0151	7
24BDS0170	1
24BDS0200	2
24BDS0214	3
24BDS0296	4
24BDS0331	5
24BDS0343	6
24BDS0354	7
24BDS0368	1
24BDS0386	2
24BDS0393	3
24BDS0403	4
24BDS0409	5
24BDS0410	6
24BDS0414	7